

eleVADR

Windows Installation and User Guide

From a clean Windows workstation to first analysis

Purpose. This guide walks an operator through installing, starting, signing in to, and using the current eleVADR local Windows platform. It is written for the newer multi-user workflow exercised in the eleVADR project, including local MongoDB, account authentication, saved reports, retained PCAPs, and reusable analysis evidence.

Important version note. The older repository README describes a container-oriented upstream workflow using docker compose up --build. The newer local Windows platform developed in this project uses the root-level first-run/start/stop scripts described below. Use the scripts present in your current eleVADR package as the authority for that package.

Audience	Windows analysts, evaluators, and local administrators
Platform	Windows 10/11, PowerShell or Command Prompt
Guide scope	Local single-workstation deployment
Validated workflow	Python 3.12.x, Node 22.23.x, npm 11.x, Docker Desktop

1. What you need before installation

Install the following before running eleVADR for the first time. The first-run preflight checks these dependencies and stops if a required component is missing or incompatible.

Component	Recommended / validated	Why eleVADR needs it
Python	3.12 or newer for the current platform package	Backend, analysis engine, platform services
Node.js	22.x (validated with 22.23.3)	Frontend toolchain
npm	Installed with Node (validated with 11.19.0)	Frontend dependency installation
Docker Desktop	Current Docker Desktop with engine running	Local MongoDB and Zeek fallback/runtime
Disk space	Several GB free, plus space for retained PCAPs	Dependencies, MongoDB, capture/report storage

Verify the tools

Open PowerShell and run:

```
python --version
node --version
npm --version
docker --version
docker info
```

Docker Desktop must be running. A successful `docker info` is a useful confirmation that the engine is available.

2. Extract eleVADR

1. Obtain the current eleVADR release/package ZIP.
2. Extract it to a normal writable folder. A short path such as `C:\eleVADR` is easiest, although a Desktop folder also works.
3. Open PowerShell in the extracted eleVADR root - the directory containing the root-level `.bat` files.

Do not run the application directly from inside the ZIP archive. Extract it first.

3. First-run setup

The current Windows platform provides a first-run script that performs the clean-machine setup sequence.

```
.\first_run_elevadr.bat
```

The script should perform four major stages: install platform Python dependencies, install frontend dependencies from the lock file, prepare/start the local MongoDB service, and bootstrap the first administrator account if no account exists.

Administrator bootstrap

On a new installation you will be prompted for an administrator username and password. Choose a strong password and retain it securely. If the database already contains a user, the script reports that first-admin bootstrap is not required instead of creating a duplicate account.

A successful run ends with a message similar to **PASS: eleVADR first-run setup completed** and directs you to start the application with `start_elevadr.bat`.

What first-run creates

The platform initializes its local configuration, starts a localhost-only MongoDB container, installs the platform Python packages, installs the frontend packages, and enables authentication after an administrator exists. Treat the generated platform environment/configuration file as deployment configuration; do not publish secrets from it.

4. Start and stop eleVADR

Start

```
.\start_elevadr.bat
```

The launcher starts the backend and frontend in separate command windows. It also checks the Zeek runtime. If native Zeek is unavailable, the launcher can use Docker as a fallback when Docker Desktop is running.

The current local frontend normally opens on `http://127.0.0.1:5173/`. The local reference backend listens on port 8765. Keep the backend and frontend command windows open while using eleVADR.

Stop

```
.\stop_elevadr.bat
```

Use the stop script when you are finished. It closes the eleVADR frontend/backend windows and terminates a remaining process listening on backend port 8765.

5. Sign in and understand the workspace

Open the frontend in a browser. When authentication is enabled, the welcome screen presents the eleVADR sign-in form. Enter the administrator or analyst credentials created for you.

After sign-in, the workspace navigation provides **Dashboard, Reports, PCAPs, Audit/Activity, Storage, and Account & Users/Account**. The exact labels and administrative functions depend on the account role.

Role	Typical capability
Admin	Analyze captures, manage Context/Modules, view platform audit/storage, create/manage local users.
Analyst	Analyze captures and work with the user's reports/PCAPs; no administrator-only user management.
Read only	Review permitted saved data; upload, analysis, re-analysis, Context changes, and destructive actions are restricted.

6. Analyze a PCAP

1. Sign in and return to the **Dashboard**.
2. Use the report-source upload area to select or drag in a .pcap or .pcapng file.
3. Review the **Detection Context** before analysis. Context represents site-specific policy and expectations such as network scope, assets, expected communications, and OT/Purdue information.
4. Review **Modules** if you need to enable/disable detection modules. The standard registry contains 75 detector modules in the validated project checkpoint.
5. Start the analysis. eleVADR runs packet/Zeek evidence extraction and the selected detection modules, then builds the report.
6. When complete, review the long-form report: **Summary, Findings, Devices, Services, and Connections**. Use the left navigation and report search/pivot controls to investigate.

Evidence reuse: the current design is intended to avoid unnecessarily running Zeek twice. Retained evidence from the capture can be reused by detector analysis and later re-analysis with a different Detection Context.

7. Work with reports

Saved Reports

Completed PCAP analyses are saved automatically for the signed-in user. Open **Reports** to search and sort saved reports. Select the report name/details area to open a report. The adjacent icons provide rename and delete actions when the account has write permission.

Inside a report

- Use the left report navigation to move between Summary, Findings, Devices, Services, and Connections.
- Use Search/pivots to filter the report by entities such as IP, service, port, subnet, manufacturer, or risk.
- Use Customize to control visible report sections.
- Use Print or Export for output. Export can create a filtered derivative view; the source report is not modified.
- Use Modules or Context to change analysis configuration. If the configuration changes while a report is open, eleVADR prompts you to generate an updated report.

8. Retained PCAPs and re-analysis

Open **PCAPs** to see captures retained for your account. Each row shows capture metadata and the number of saved analyses linked to that capture.

- History icon: show/hide prior analyses for the retained capture.
- Re-analyze icon: run analysis again using the currently active Detection Context/module selection without uploading the PCAP again.
- Delete icon: remove the retained capture, subject to permissions and confirmation.
- Analysis history can be used to select two analyses from the same capture and compare findings, assets, services, connections, Context changes, module changes, and runtime changes.

9. Detection Context and Modules

Detection Context

Context is where eleVADR learns what is expected for the capture/site. Complete as much of the wizard as is known. Incorrectly marking normal site behavior as unknown or unexpected can increase findings; overly broad allowlisting can suppress useful detections.

Detection Modules

Modules control which detector families execute. Leave the standard selection intact unless you have a reason to tailor analysis. When reviewing a finding, use its provenance/explainability information to understand which Zeek log type and supporting fields contributed to the result.

10. Account, users, storage, and audit

Account

Use **Account** to change your password. Administrators also see **Account & Users**, where they can create users, assign roles, disable/enable accounts, and review account status.

Storage

The Storage view summarizes retained capture storage, report storage, retention policy, and configured limits. Administrators can review aggregate usage and run cleanup for expired captures and, when requested, orphaned capture files.

Audit / Activity

Administrators can review platform-wide audit activity. Non-admin users see activity scoped to their account where supported. Use this view when investigating account actions, storage cleanup, analysis/re-analysis, and other platform events.

11. Validate the installation

For a release/development checkout that includes the validation scripts, run the platform security/regression gate from the repository root:

```
.\run_platform_regression_tests.bat
```

A healthy checkpoint should complete with all tests passing. The exact test count can increase as the project evolves, so treat the PASS/FAIL result - not a hard-coded count - as authoritative.

The broader project also includes regression/release-candidate validation scripts. These are primarily maintainer/developer checks rather than normal operator startup steps.

12. Troubleshooting

Symptom	What to check
Docker/MongoDB setup fails	Start Docker Desktop; verify docker info succeeds; retry first-run.
PCAP analysis cannot start	Verify Docker is running or a native/explicit Zeek runtime is available. JSON report viewing can work even when PCAP analysis cannot.
Frontend dependencies fail	Use the package lock and the supported Node/npm versions. Avoid --force or --legacy-peer-deps unless the project intentionally documents that choice.
Python package conflicts	A dedicated Python virtual environment is preferable for isolation. If using a shared user Python installation, unrelated packages can make pip check report conflicts that are not eleVADR dependencies.
Port 8765 already in use	The start script attempts to stop a stale listener. If necessary, close the old eleVADR backend process and retry.
Cannot sign in	Confirm the first-run bootstrap created a user, authentication is enabled, username/password are correct, and the backend window shows no startup error.
Report opens but PCAP analysis fails	Check the Zeek/Docker runtime. Loading JSON and analyzing raw PCAPs have different runtime requirements.

Symptom	What to check
UI looks stale after replacing files	Stop/restart the frontend. If needed, refresh the browser without cache after rebuilding/restarting.

13. Clean-install quick checklist

- 1. Install Python, Node/npm, and Docker Desktop.
- 2. Start Docker Desktop and verify docker info.
- 3. Extract the current eleVADR package.
- 4. Open PowerShell in the eleVADR root.
- 5. Run `.\first_run_elevadr.bat`.
- 6. Create the first administrator when prompted (new install only).
- 7. Confirm first-run reports PASS.
- 8. Run `.\start_elevadr.bat`.
- 9. Open `http://127.0.0.1:5173/`.
- 10. Sign in.
- 11. Review Context and Modules, then analyze a PCAP.
- 12. Confirm the resulting report is saved and the PCAP appears under Retained PCAPs.
- 13. When finished, run `.\stop_elevadr.bat`.

14. Source/version notes

This guide intentionally follows the newer Windows platform workflow exercised during the eleVADR project work. The packaged repository README available in the supplied source bundle still documents an older/upstream container-first path (`docker compose up --build`) and manual host development prerequisites. If a future release changes the root-level scripts, follow the scripts and release notes shipped with that release.

The supplied source package also contains `eleVADR_detection_module_reference.pdf`, which is the companion reference for detector-specific behavior.